

Agentic Hallucination in logical context

Anshika Jain

Mithilesh Kumar

August 5, 2026

Chapter 1

Lean 4 Environment Setup

Agentic Hallucinations — Independent Study. Target OS: Windows.

This document walks through installing Lean 4, connecting it to Mathlib, and verifying a working end-to-end setup. It also explains the pieces (`elan`, `lake`, `lean-toolchain`) you'll be switching between constantly once the agent↔Lean harness is running.

1.1 What you're installing

Three components, each with a distinct job:

Component	Role
<code>elan</code>	Version manager. Installs and switches Lean toolchain versions automatically per-project (reads <code>lean-toolchain</code>). Analogous to <code>nvm</code> for Node or <code>pyenv</code> for Python.
<code>lake</code>	Lean's build tool and package manager (bundled with <code>elan</code>). Creates projects, resolves dependencies, builds, and runs executables. Analogous to <code>cargo</code> or <code>npm</code> .
Lean 4 VS Code extension	Editor integration. Talks to the Lean language server, renders the goal state (Infoview), shows diagnostics inline.

You do **not** install “Lean” directly — `elan` installs and manages the actual `lean` compiler versions on demand, per project.

1.2 Prerequisites

Open Command Prompt and confirm Git and curl are present:

```
git --version
curl --version
```

If either errors, install [Git for Windows](#) (accept defaults; choose VS Code as Git's default editor when prompted) and reopen the terminal.

Install [VS Code](#) if you don't have it. In the installer, check “**Add to PATH**” and “**Register Code as an editor for supported file types.**”

1.3 Install elan

Open a new terminal (PowerShell or Command Prompt) and run:

```
curl -O --location https://elan.lean-lang.org/elan-init.ps1
powershell -ExecutionPolicy Bypass -f elan-init.ps1
```

When prompted, choose **1** for the default installation. Then clean up and restart your terminal:

```
del elan-init.ps1
exit
```

Open a fresh terminal and confirm:

```
elan --version
lake --version
```

Both should print version numbers. If **lake** is not recognized, close and reopen the terminal (elan modifies your **PATH**, which only takes effect in new shells) — or restart your machine if that doesn't resolve it.

1.4 Install the Lean 4 VS Code extension

In VS Code: **Ctrl+Shift+X** → search **lean4** → install the official extension published by **leanprover**. Choose “Trust Publisher & Install” if prompted. This opens the **Lean 4 Setup Guide**, which double-checks the above steps automatically — worth glancing at, but everything in it is covered here.

Sanity check:

1. **Ctrl+N** → new file → **Ctrl+S** → save as **Test.lean** anywhere.
2. VS Code will silently download a full Lean toolchain in the background (a progress popup appears bottom-right; the Infoview is unavailable until this finishes — can take several minutes the first time).
3. Type **#eval 1 + 1** into the file.
4. Hover over **eval** — you should see a tooltip showing **2**, with no red squiggles.

If that works, Lean itself is installed correctly.

1.5 Create a project

Everything in Lean lives inside a project — a folder with a **lakefile.toml/lakefile.lean** (dependency + build config) and a **lean-toolchain** file (pins the Lean version). A loose **.lean** file outside a project will throw confusing errors, especially once you **import** anything.

Two project types you'll use:

A. Plain project (no Mathlib) — for isolated tactic experiments:

```
lake new MyLeanProject
cd MyLeanProject
lake build
```

B. Mathlib-dependent project — for anything touching real math (topology, algebra, set theory — i.e. your actual problem set):

```
lake +leanprover-community/mathlib4:lean-toolchain new MyMathlibProject math
cd MyMathlibProject
lake build
```

The `math` keyword adds Mathlib as a dependency so `import Mathlib` works. First build after this can take a while — it’s fetching Mathlib’s dependency graph.

1.5.1 Working in an already-cloned repository

Everything above (`lake new ... math`) is for starting a project from scratch. Joining a project that already exists — this one, for instance — is a different situation: the repository already carries its own `lakefile.toml` and `lean-toolchain`, so you skip `lake new/lake init` entirely and simply clone it.

```
git clone https://github.com/drmithileshkumar/agentik.git
cd agentik
```

What to refresh. A fresh clone has no `.lake/` directory — it’s gitignored, and entirely regenerable, so it is never part of what you clone. Refreshing it means fetching the prebuilt Mathlib cache and building:

```
lake exe cache get
lake build
```

Note that `lake update` is **not** needed here. That command resolves dependencies to newer versions — a different operation, covered below in “Keeping things updated.” A first clone should build against exactly what `lake-manifest.json` already pins.

A lake gotcha worth knowing. `lake init <package-name> <template>` takes the package name first and the template second (`std/exe/lib/math`). If the folder is literally named `math`, running `lake init` on its own parses `math` as the package *name* with no template given — and you silently get a plain `std` project rather than the Mathlib-enabled one you wanted. Keep the project folder’s name distinct from `math`, and always pass the template explicitly:

```
lake init myproject math
```

Fetch the prebuilt Mathlib cache (do this — avoids compiling Mathlib from source, which takes hours):

```
lake exe cache get
```

Open the project in VS Code: `code .` from inside the folder, or `File > Open Folder`. Click “**Trust authors**” when prompted.

1.6 Verify Mathlib is wired up

Create `MyMathlibProject/MyMathlibProject/Test.lean`:

```
import Mathlib.Topology.Basic

#check TopologicalSpace
```

With the cursor on the last line, the Infoview (right-hand panel; restore with `Ctrl+Shift+Enter` if it's hidden) should show:

```
TopologicalSpace.{u} (α : Type u) : Type u
```

No errors → the environment is fully wired: `elan` → `lake` → `Mathlib` → VS Code language server → Infoview.

1.7 Leanblueprint set up

Leanblueprint turns a LaTeX blueprint into a website with a dependency graph, plus a PDF.

1.7.1 Installing it

This repository already has this done — the virtual environment and `leanblueprint` were added at commit `d2ed81a` — so the instructions below are for reference, or for reproducing the setup elsewhere, rather than something you need to redo in this checkout.

Install it into a project-local virtual environment rather than globally:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install leanblueprint
```

Add `.venv/` to `.gitignore` — it's a local environment, not project content.

1.7.2 Two dependencies for a full local build

Beyond the Python package, a *full local* build needs two pieces of system software:

- **Graphviz** — renders the dependency graph, via the `dot` and `tred` binaries.
- **A TeX distribution** (MiKTeX or TeX Live) — needed for the PDF, and also for `kpsewhich`, which the HTML path uses to locate files. So this is required even if you only ever build the website.

Neither is installed by `pip`. CI already has both, so pushing and letting the workflow build is a reliable fallback if you'd rather not install them locally.

1.7.3 How blueprint/src is organized

`content.tex` assembles the blueprint by `\input`-ing each chapter in order. `web.tex` and `print.tex` are the two entry points — the website and the PDF respectively — and both pull in the same `content.tex`. Individual chapters live in `blueprint/src/chapters/`.

1.7.4 Build commands

Run these from the `blueprint/` directory:

```
leanblueprint web          # build the website
leanblueprint pdf          # build the PDF
leanblueprint serve        # serve the website locally
leanblueprint checkdecls   # check declarations named in the blueprint exist in Lean
```

1.8 Project anatomy (what’s actually on disk)

```
MyMathlibProject/  
├─ lakefile.toml           # dependencies, build targets  
├─ lean-toolchain          # pins exact Lean version (elan reads this)  
├─ lake-manifest.json      # resolved dependency versions (like a lockfile)  
├─ .lake/                  # build artifacts, downloaded deps –  
safe to delete/regenerate  
└─ MyMathlibProject/  
    └─ Test.lean           # your actual Lean source
```

`lean-toolchain` is the important one for environment switching: `elan` silently swaps the active Lean version based on whatever this file says, per-project, with no manual intervention. If two projects pin different Lean versions, moving between their folders in a terminal changes which `lean/lake` binary actually runs — this matters once the agent harness is juggling multiple project directories.

1.9 Command line vs. VS Code — two ways to talk to Lean

You’ll use both, for different purposes:

- **VS Code (interactive):** editing proofs by hand, reading goal states, learning tactics. This is where *you* work.
- **Terminal / `lake env lean` / `lake build`:** batch-checking a file, scripting, and — critically — how the **agent harness** will talk to Lean. The agent won’t have an Infoview; it will invoke Lean non-interactively (or via the Lean REPL / language server protocol) and parse structured output. Understanding what VS Code is doing under the hood *is* understanding the harness.

Run a single file from the command line to see raw output (no pretty Infoview):

```
lake env lean MyMathlibProject/Test.lean
```

No output = success. Errors print as plain text — this is closer to what the agent will actually receive.

1.10 Keeping things updated

```
curl -L https://raw.githubusercontent.com/leanprover-community/mathlib4/master/lean-  
toolchain -o lean-toolchain  
lake update  
lake exe cache get  
lake build
```

Run this inside a project when Mathlib has moved on and you want the latest version.

Note: this section is for pulling a newer Mathlib version into a project you’re already working in. If you’ve just cloned the repository for the first time, you don’t need `lake update` — see “Working in an already-cloned repository” above instead.

1.11 Troubleshooting

curl: (35) schannel: next InitializeSecurityContext failed during `lake exe cache get` — usually antivirus intercepting downloads. Disable it temporarily and re-run with the `force` flag:

`lake exe cache get!`

uncaught exception: no such file or directory (error code: 2) — a corrupted toolchain download. Fix:

1. Delete the offending folder under `C:\Users\<you>\.elan\toolchains`
2. Delete the matching file under `C:\Users\<you>\.elan\update-hashes`
3. Re-run `lake build` — it re-downloads automatically.

Disk cleanup — safe to delete: `.lake/` inside any project (regenerates on build), old entries in `~/.elan/toolchains`, and `~/.cache/mathlib`.

1.12 Next up

With this environment working, the next docs cover: core tactics (interactive), a tactic cheat sheet, a full tutorial, and the agent↔Lean pipeline design that reuses the command-line invocation pattern from the command-line section above.

1.13 References

- [Install — Lean Lang \(recommended VS Code path\)](#)
- [Manual Installation — Lean Lang \(Windows/macOS/Linux CLI steps\)](#)
- [Lean projects — Lean Community \(project creation, Mathlib cache, troubleshooting\)](#)
- [elan \(Lean version manager\) — GitHub](#)
- [Lean 4 VS Code extension manual](#)

See `docs/resources.md` (the project bibliography, formerly `00_resources.md`) for the full project bibliography.

Chapter 2

Lean 4 Tutorial

Agentic Hallucinations — Independent Study. A from-scratch tutorial in tactic-mode proof writing, building toward the topic-segmented problem set (set theory, topology, algebra) this project will run the agent against.

Primary sources: lean4.dev/tactics (course + per-tactic pages), [Theorem Proving in Lean 4, Mathematics in Lean](#) (Avigad & Massot), [Mathlib4 tactic docs](#). Full annotated bibliography: [docs/resources.md](#). Environment setup assumed complete — see the Lean 4 Environment Setup chapter.

Table of contents

1. How Lean thinks: goals, context, term vs. tactic mode
2. Core tactics I — closing goals (`exact`, `apply`, `assumption`, `rfl`)
3. Core tactics II — rewriting (`rw`, `simp` and its variants)
4. Core tactics III — introducing and structuring (`intro`, `have`, `let`, `suffices`)
5. Case analysis and induction (`cases`, `induction`, `rcases`, `obtain`)
6. Automation and decision procedures (`omega`, `ring`, `linarith`, `nlinarith`, `decide`, `aesop`)
7. Structuring proofs (`calc`, `by_contra`, `by_cases`, `contrapose`, `push_neg`)
8. Equality machinery (`ext`, `funext`, `congr`, `symm`, `trans`)
9. Tactic combinators (`<`, `>`, `·`, `first`, `repeat`, `all_goals`)
10. Worked examples — Set theory
11. Worked examples — Topology
12. Worked examples — Algebra
13. Reading errors and diagnosing failed proofs
14. From tactics to the agent pipeline
15. Mathlib conventions: naming, dot notation, anonymous constructors
16. Exercises (with worked solutions)

2.1 How Lean thinks: goals, context, term vs. tactic mode

Every Lean proof is, underneath, a **term** — a value whose type is the proposition being proved. `theorem add_comm' (a b : Nat) : a + b = b + a := Nat.add_comm a b` is a complete proof written directly as a term: `Nat.add_comm a b` is a value, and its type happens to be `a + b = b + a`.

Writing proof terms directly gets unwieldy fast, so almost all real proofs use **tactic mode**, entered with `by`:

```
theorem add_comm' (a b : Nat) : a + b = b + a := by
  exact Nat.add_comm a b
```

Inside `by`, you're not writing a term — you're issuing *tactics*, each of which transforms a **goal state** into a new (possibly empty) goal state. A goal state looks like:

```
a b : Nat
⊢ a + b = b + a
```

Above `⊢` is the **local context**: every variable and hypothesis currently in scope. After `⊢` is the **goal**: the proposition still to be proved. A tactic either closes the goal (context becomes empty — done), transforms it into a different goal, or splits it into multiple goals (e.g. one per case of an **induction**).

This is exactly the loop this project automates: an LLM agent looks at a goal state, proposes one tactic, and Lean reports back either a new goal state or an error. **Everything in this tutorial is really a catalog of the “moves” the agent has available at each step**, which is why the taxonomy of *which* move it picks — and how often it picks a wrong one — is the statistical object of interest for the hallucination analysis (see `docs/resources.md §5`).

A style note before we start: Lean tactic mode is whitespace-sensitive in a specific way — tactics separated by newlines run in sequence on the current (first) goal; `·` (a centered dot, typed `\.`) explicitly focuses a single goal so that indented tactics under it apply only there. You'll see this constantly once goals start splitting (§5 onward).

2.2 Core tactics I — closing goals

2.2.1 `rfl` — reflexivity

Closes a goal `a = a` up to Lean's notion of *definitional* equality — not just syntactic identity, but anything that reduces to the same normal form by unfolding definitions and computing.

```
example : 2 + 2 = 4 := by rfl
example (n : Nat) : n = n := by rfl
```

`rfl` fails as soon as the two sides aren't definitionally equal, even if they're *provably* equal by some other route — that's when you need `rw`, `simp`, `ring`, or `omega` instead.

2.2.2 `exact` — direct proof

`exact e` closes the goal by supplying a term `e` whose type matches exactly.

```
example (h : 2 + 2 = 4) : 2 + 2 = 4 := by exact h
example (n : Nat) : n + 0 = n := by exact Nat.add_zero n
```

If you don't know the exact lemma name, `exact?` searches the library and suggests one:

```
example (n : Nat) : n + 0 = n := by exact?
-- Suggests: exact Nat.add_zero n
```

2.2.3 `apply` — working backwards

`apply e` unifies the *conclusion* of `e`'s type with the goal, and turns any remaining *premises* of `e` into new goals. Where `exact` needs everything handed to it, `apply` lets Lean fill in what it can infer and leaves the rest for you.

```
example (a b c : Nat) (hab : a = b) (hbc : b = c) : a = c := by
  apply Eq.trans
  · exact hab
  · exact hbc
```

`Eq.trans : a = b → b = c → a = c` doesn't fully match the goal `a = c` on its own — `apply` inserts a metavariable for the missing middle term and creates two new goals for the two premises. This is exactly why `apply` sometimes produces goals that look stranger than the original: it's exposing structure the lemma needed that wasn't visible in your one-line goal.

`apply?` is the library-search analog for `apply`.

2.2.4 `assumption`

Scans the local context for a hypothesis whose type matches the goal exactly, closing it if found.

```
example (P Q : Prop) (hp : P) (hq : Q) : P := by assumption -- finds hp
```

2.2.5 Quick reference

Tactic	Use when
<code>exact e</code>	You have exactly what's needed
<code>apply lemma</code>	Lemma's <i>conclusion</i> matches; premises become new goals
<code>assumption</code>	Goal matches a hypothesis, don't want to name it
<code>rfl</code>	Both sides compute to the same value
<code>exact? / apply?</code>	You don't know the lemma name

2.3 Core tactics II — rewriting

2.3.1 `rw` — precise, one substitution at a time

Given `h : a = b`, `rw [h]` finds `a` in the goal and replaces every occurrence with `b`.

```
example (a b : Nat) (h : a = b) : a + 1 = b + 1 := by
  rw [h] -- goal becomes b + 1 = b + 1
```

Note this closes automatically — `rw` calls `rfl` after rewriting if that finishes the goal. Direction matters: `rw [h]` goes left-to-right (per `h`'s statement); `rw [← h]` (type `\l` or `\leftarrow` for `←`) goes right-to-left.

```
example (a b : Nat) (h : a + 1 = b) : b = a + 1 := by
  rw [← h] -- replaces b with (a + 1); goal becomes a + 1 = a + 1
```

Chain multiple rewrites in one call — they apply left to right in sequence:

```
example (a b c : Nat) (h1 : a = b) (h2 : b = c) : a = c := by
  rw [h1, h2]
```

Rewrite inside a hypothesis instead of the goal with `at`:

```
example (a b : Nat) (h1 : a = b) (h2 : a + 1 = 5) : b + 1 = 5 := by
  rw [h1] at h2
  exact h2
```

Common trap: `rw` replaces *all* syntactic occurrences of the LHS pattern, which is occasionally more than you want (`rw [h]` on `a + a = 10` with `h : a = b` gives `b + b = 10`, not just one side). Use `nth_rewrite n [h]` to target a specific occurrence.

2.3.2 `simp` — automated simplification

Where `rw` applies rules you name, `simp` repeatedly applies a large database of lemmas tagged `@[simp]` until nothing more simplifies, and frequently closes the goal outright.

```
example (n : Nat) : n + 0 + 0 + 0 = n := by simp
example (xs : List Nat) : xs ++ [] = xs := by simp
```

Feed it extra lemmas or local hypotheses in brackets:

```
example (a b : Nat) (h : a = 5) : a + b = 5 + b := by simp [h]
```

`simp` is not omniscient. It's a simplifier, not a general prover — it won't discover a proof strategy, only apply known-good rewrite rules to a fixed point. `a * (b + c) = a*b + a*c` is *not* closed by bare `simp` (distributivity isn't tagged `@[simp]`); that's a job for `ring`.

Four variants worth knowing cold:

Variant	Behavior
<code>simp only [l1, l2, ...]</code>	Uses only the listed lemmas — reproducible, doesn't silently change behavior if the global <code>simp</code> set grows. Preferred in anything meant to be stable.
<code>simp_all</code>	Simplifies the goal and every hypothesis , using each hypothesis as an extra rewrite rule. Powerful for messy contexts, but harder to predict.
<code>simp?</code>	Runs <code>simp</code> , then prints the minimal <code>simp only [...]</code> that achieves the same result — the standard way to convert an exploratory <code>simp</code> into a reproducible one.
<code>dsimp</code>	<i>Definitional</i> simplification only — unfolds/computes, never applies a proved rewrite lemma. Never fails unexpectedly, but also never does as much.

```
example (n : Nat) : n + 0 = n := by
  simp?
  -- Try this: simp only [Nat.add_zero]

  simp at h / simp at * target hypotheses or everything at once, exactly like rw's at.
```

2.3.3 norm_num, norm_cast, push_cast

`norm_num` proves/simplifies goals about concrete numerals ($21 - 6 = 15$). `norm_cast`/`push_cast` normalize numeric-type coercions (moving between `, , ,`) — essential once problems mix integer and real arithmetic, common in the topology/analysis parts of the problem set.

2.4 Core tactics III — introducing and structuring

2.4.1 intro — assuming hypotheses

When the goal is an implication $P \rightarrow Q$ or a universal $\forall x, P\ x$, `intro` moves the antecedent/quantified variable into the context.

```
example (P Q : Prop) : P → Q → P := by
  intro hp hq
  exact hp
```

```
example : ∀ n : Nat, n = n := by
  intro n
  rfl
```

`intro` can destructure as it introduces:

```
example (P Q : Prop) : P ∧ Q → Q ∧ P := by
  intro ⟨hp, hq⟩
  exact ⟨hq, hp⟩
```

Prefer explicit names (`intro n hn`) over the bare `intros`, which auto-generates opaque names like `a†` — fine for quick exploration, bad for anything you or an agent needs to refer back to later in the proof.

2.4.2 have and let — intermediate steps

`have h : T := proof` states and proves an intermediate fact, adding $h : T$ to the context. The *value* is opaque afterward — only its type matters to the rest of the proof.

```
example (a b : Nat) (ha : a > 5) (hb : b > 3) : a + b > 8 := by
  have hab : a + b > 5 + 3 := by omega
  omega
```

`let x := val` introduces a **transparent** local definition — its value, not just its type, is visible to later tactics like `rfl`.

```
example : 10 = 5 + 5 := by
  let x := 5
  rfl -- works because x unfolds to 5
```

Use `have` for propositions/proofs you'll reference by type; use `let` for computed values you want to reuse and potentially unfold later.

2.4.3 suffices — working backwards from a stronger claim

`suffices h : T by tac` replaces the goal with `T`, where `tac` discharges the *original* goal assuming `h : T`.

```
example (n : Nat) : n + n ≥ n := by
  suffices h : n ≤ n + n by omega
  omega
```

Read it as: “it suffices to show `T`; here’s how `T` implies what I actually wanted (`by tac`); now let me prove `T`.”

2.4.4 show — restating the goal

`show T` changes the displayed goal to a definitionally-equal `T` — purely for clarity/debugging, doesn’t change what’s actually being proved.

2.5 Case analysis and induction

2.5.1 cases — splitting on constructors

`cases X` produces one goal per constructor of `X`’s type, replacing `X` with that constructor’s shape in each branch.

```
example (b : Bool) : b = true ∨ b = false := by
  cases b with
  | true => left; rfl
  | false => right; rfl
```

```
example (n : Nat) : n = 0 ∨ n > 0 := by
  cases n with
  | zero => left; rfl
  | succ m => right; omega
```

`cases` also works on propositions built from inductive types — `Or` has two constructors (`inl/inr`), `And` has one with two fields:

```
example (P Q R : Prop) (h : P ∨ Q) (hpr : P → R) (hqr : Q → R) : R := by
  cases h with
  | inl hp => exact hpr hp
  | inr hq => exact hqr hq
```

2.5.2 induction — proof by mathematical induction

`induction x with` produces a base case and, for each recursive constructor, a step case that includes an **induction hypothesis** `ih`.

```
theorem add_zero' (n : Nat) : n + 0 = n := by
  induction n with
  | zero => rfl
  | succ m ih =>
```

```
-- ih : m + 0 = m
-- goal : (m + 1) + 0 = m + 1
simp [ih]
```

The induction hypothesis is the entire point — it’s the license to *assume* the property holds for the smaller case while proving it for the larger one. Forgetting to use `ih` (proving the step case “from scratch”) is a common failure mode both for beginners and, notably, for agents that don’t track what’s actually available in context.

2.5.3 `rcases / obtain` — recursive destructuring (Mathlib)

`cases` gets unwieldy once structures nest (an `inside an inside an ...`). `rcases/obtain` (both require `import Mathlib.Tactic.Cases` / are available under `import Mathlib.Tactic`) destructure all of that in one pattern.

```
example (h : ∃ n m : Nat, n < m ∧ m < 10) : True := by
  rcases h with ⟨n, m, hnm, hm10⟩
  trivial
```

```
example (h : ∃ n : Nat, n > 0 ∧ n < 10) : True := by
  obtain ⟨n, hn_pos, hn_lt⟩ := h
  trivial
```

Pattern language: `⟨p1, p2⟩` destructs pairs// ; `p1 | p2` splits ; `rfl` substitutes an equality the instant it’s introduced (extremely handy — see below); `_` discards a part you don’t need.

```
example (h : ∃ n : Nat, n = 5) : 5 = 5 := by
  rcases h with ⟨_, rfl⟩ -- n is substituted by 5 everywhere
  rfl
```

`rintro` fuses `intro` with an `rcases` pattern: `rintro ⟨a, b⟩ ⟨c, d⟩` introduces and destructures two arguments in one line.

cases/induction vs. rcases/obtain: the former are core-Lean and exhaustive (they force you to name every constructor case); the latter are Mathlib conveniences layered on top, better for existentials and nested nonsense, worse for anything where you actually need the inductive structure (like a genuine induction).

2.6 Automation and decision procedures

These tactics are *decision procedures* or *heuristic search* over specific fragments of mathematics — when your goal falls in their fragment, they either succeed or definitively fail (no partial credit, no “almost”).

2.6.1 `omega` — linear arithmetic over /

Handles `+`, `-`, comparisons, and multiplication/division/modulo **by constants** (not variable $\times$ variable).

```
example (n : Nat) : n ≤ n + 1 := by omega
example (a b c : Nat) (h1 : a ≤ b) (h2 : b < c) : a < c := by omega
example (n : Nat) : n / 2 ≤ n := by omega
example (n : Nat) (h : n % 2 = 0) : 2 * (n / 2) = n := by omega
```

`omega` automatically pulls in every relevant hypothesis in context — you never tell it which ones to use. It correctly understands `Nat`’s truncated subtraction ($a - b = 0$ when $a < b$) and `Int`’s standard subtraction, which is a frequent source of confusion when porting a proof between the two.

It fails immediately on anything nonlinear:

```
-- example (x y : Nat) : x * y = y * x := by omega    -- fails: nonlinear
example (x y : Nat) : x * y = y * x := by ring        -- use ring instead
```

2.6.2 `ring / ring_nf` — polynomial identities

Proves equalities that follow purely from ring axioms (associativity, commutativity, distributivity) — no need to spell out the expansion by hand.

```
example (x y : Int) : (x + y)^2 = x^2 + 2*x*y + y^2 := by ring
example (a b c : Int) : (a + b + c)^2 = a^2+b^2+c^2 + 2*a*b + 2*b*c + 2*a*c := by ring
```

`ring` cannot handle division (use `field_simp` first) or inequalities (use `nlinarith`) or anything involving an opaque function. `ring_nf` normalizes an expression to canonical form *without* requiring it close the goal — useful mid-proof.

2.6.3 `linarith / nlinarith` — linear / mildly-nonlinear inequalities

`linarith` proves linear (in)equalities from hypotheses over ordered fields, by finding a linear combination that derives a contradiction from the negated goal. `nlinarith` extends it with limited nonlinear preprocessing (it can use provided facts like squares being nonnegative):

```
example (x : Int) : x * x ≥ 0 := by nlinarith [sq_nonneg x]
```

2.6.4 `decide / native_decide`

Evaluate a `Decidable` proposition directly to `true/false`. `decide` runs inside the kernel (slow, fully trusted); `native_decide` compiles to native code first (fast, relies on trusting the compiler — appropriate for larger finite checks, not for anything safety-critical in the proof).

2.6.5 `aesop` — general automated search

`aesop` searches over a registered rule set (`@[aesop]`-tagged lemmas plus built-ins) trying to close the goal by whatever combination works. `aesop?` prints the proof it found, which is the standard way to turn an `aesop` call into a reproducible, auditable tactic sequence.

2.6.6 Choosing among them

Goal shape	Tactic
Concrete numeral computation	<code>norm_num</code>
Linear arithmetic, /	<code>omega</code>
Polynomial equality	<code>ring</code>
Equality with division	<code>field_simp</code> then <code>ring</code>
Linear inequalities	<code>linarith</code>
Nonlinear inequalities	<code>nlinarith</code> (feed it hints)
Decidable finite check	<code>decide / native_decide</code>
“Try everything reasonable”	<code>aesop</code>

2.7 Structuring proofs

2.7.1 `by_contra` — proof by contradiction

`by_contra h` turns a goal P into a goal `False`, adding $h : \neg P$ to the context.

```
example (P : Prop) (h :  $\neg P \rightarrow \text{False}$ ) : P := by
  by_contra hP
  exact h hP
```

2.7.2 `by_cases` — splitting on a proposition

`by_cases h : P` produces two goals: one with $h : P$, one with $h : \neg P$.

```
example (n : Nat) : n = 0  $\vee$  n  $\neq$  0 := by
  by_cases h : n = 0
  · exact Or.inl h
  · exact Or.inr h
```

Both tactics rely on classical reasoning (excluded middle) — standard throughout Mathlib, but worth knowing if a proof needs to stay constructive.

2.7.3 `contrapose` / `push_neg`

`contrapose` turns a goal $P \rightarrow Q$ into $\neg Q \rightarrow \neg P$; `contrapose!` additionally runs `push_neg` on the result. `push_neg` on its own pushes a $\neg$ inward through quantifiers and connectives via De Morgan's laws — e.g. turning $\neg \forall x, P\ x$ into $\exists x, \neg P\ x$, or $\neg(a < b)$ into $b \leq a$. Both are usually paired with `by_contra` or `by_cases` to get a negated hypothesis into a *usable* shape rather than a raw $\neg(\dots)$ blob.

2.7.4 `calc` — calculational proofs

Chains a sequence of relational steps, each independently justified, reading like a proof on paper.

```
example (a b c d : Nat) (h1 : a = b) (h2 : b < c) (h3 : c  $\leq$  d) : a < d := calc
  a = b := h1
  _ < c := h2
  _  $\leq$  d := h3
```

Each `_` refers to the previous line's right-hand side. `calc` can mix relations ($=$, $<$, $\leq$, $\neq$) as long as they compose transitively, and any tactic can justify a step:

```
example (x y : Int) : (x + y)^2 = x^2 + 2*x*y + y^2 := calc
  (x + y)^2 = (x + y) * (x + y) := by ring
  _ = x*x + x*y + y*x + y*y := by ring
  _ = x^2 + 2*x*y + y^2 := by ring
```

`calc` is worth reaching for whenever a proof's *legibility* matters as much as its correctness — which, notably, includes anywhere you'd want a human (or a later analysis pass) to audit *why* the agent believed a step, not just *that* Lean accepted it.

2.8 Equality machinery: `ext`, `funext`, `congr`, `symm`, `trans`

2.8.1 `funext` and `ext` — extensionality

Two functions are equal exactly when they agree everywhere; two sets are equal exactly when they have the same elements. `funext x` reduces $f = g$ to $f\ x = g\ x$; `ext` (Mathlib, more general) applies whatever extensionality lemma fits the goal's type — functions, sets, structures, products.

```
example : (fun x : Nat => x + 0) = (fun x => x) := by
  funext x
  simp
```

```
example (A B : Set Nat) (h : ∀ x, x ∈ A ↔ x ∈ B) : A = B := by
  ext x
  exact h x
```

```
example (A B : Set α) : A ∩ B = B ∩ A := by
  ext x
  constructor
  · intro ⟨ha, hb⟩; exact ⟨hb, ha⟩
  · intro ⟨hb, ha⟩; exact ⟨ha, hb⟩
```

Marking a custom structure `@[ext]` auto-generates an extensionality lemma the `ext` tactic can then use.

2.8.2 `congr` — from function equality to argument equality

`congr` reduces a goal $f\ a = f\ b$ to $a = b$ (and, for multi-argument functions, one subgoal per argument).

2.8.3 `symm` / `trans`

`symm` flips $a = b$ to $b = a$ in the goal; `trans y` splits $a = c$ into $a = y$ and $y = c$ — the tactic-mode analog of a manual `Eq.trans`/one `calc` step.

2.9 Tactic combinators

Combinators glue tactics together without you manually tracking every resulting goal:

```
-- Runs `simp` on every goal produced by `cases`
example (b : Bool) : b || true = true := by cases b <|> simp

-- Focuses exactly the first goal
example (P Q : Prop) (hp : P) (hq : Q) : P ∧ Q := by
  constructor
  · exact hp
  · exact hq
```

Combinator	Behavior
$t1 <;> t2$	Run $t2$ on every goal $t1$ produces
$\cdot tac$	Focus the first goal; tac must close it
$first \mid t1 \mid t2$	Try $t1$; on failure try $t2$
$repeat tac$	Repeat until tac fails
$all_goals tac$	Run on every goal; fails if any fails
$any_goals tac$	Run on every goal; fails only if none succeed

These matter for the agent harness specifically: a single agent-proposed “tactic” is often, in practice, a short combinator expression (`intro n; cases n <;> simp`) rather than one atomic step — which affects how we define “one iteration” in the statistical analysis (see `docs/resources.md` §5 and the Agent↔Lean 4 Pipeline chapter).

2.10 Worked examples — Set theory

Sets in Mathlib are `Set  $\alpha$` for some ambient type α ; membership is $x \in s$. The single most useful tactic here is `ext` — almost every “these two sets are equal” goal reduces to “these two sets have the same members,” which reduces to an iff you prove pointwise.

```
-- Intersection is commutative
example (A B : Set  $\alpha$ ) : A  $\cap$  B = B  $\cap$  A := by
  ext x
  simp only [Set.mem_inter_iff]
  tauto

-- Distributivity of  $\cap$  over  $\cup$ 
example (A B C : Set  $\alpha$ ) : A  $\cap$  (B  $\cup$  C) = (A  $\cap$  B)  $\cup$  (A  $\cap$  C) := by
  ext x
  simp only [Set.mem_inter_iff, Set.mem_union]
  tauto

-- Subset proofs: intro the element, chase membership
example (A B C : Set  $\alpha$ ) (h1 : A  $\subseteq$  B) (h2 : B  $\subseteq$  C) : A  $\subseteq$  C := by
  intro x hx
  exact h2 (h1 hx)

-- Existentials over sets
example (A : Set Nat) (h :  $\exists n \in A, n > 5$ ) :  $\exists n, n \in A$  := by
  obtain ⟨n, hn, _⟩ := h
  exact ⟨n, hn⟩
```

Note the pattern: `ext x` to get down to elements, `simp only [Set.mem_...]` (or plain `simp`) to unfold membership into a propositional statement, then propositional tactics (`tauto`, `constructor`, `intro/exact`) to close it. $\subseteq$ unfolds definitionally to $\forall x, x \in A \rightarrow x \in B$, so subset proofs are almost always `intro x hx; ...`

2.11 Worked examples — Topology

Mathlib’s `TopologicalSpace  $\alpha$` typeclass carries an `IsOpen : Set  $\alpha$  → Prop` predicate plus the usual axioms (arbitrary unions of opens are open, finite intersections of opens are open, the whole

space and empty set are open). A function $f : \alpha \rightarrow \beta$ between topological spaces is `Continuous` f exactly when the preimage of every open set is open — this is *the* recurring proof shape for continuity goals.

```
-- Continuity as preimage-of-open-is-open (schematic – exact lemma names
-- depend on the specific Mathlib continuity API in scope; verify with #check
-- or `exact?` before relying on this in a real proof)
example {α β : Type*} [TopologicalSpace α] [TopologicalSpace β]
  (f : α → β) (h : ∀ U, IsOpen U → IsOpen (f ⁻¹' U)) : Continuous f := by
  exact continuous_def.mpr h
```

Limits are expressed via `Tendsto` and neighborhood filters $\square$: “the limit of $2 \cdot x$ as $x \rightarrow 3$ is 6 ” is `Tendsto (fun x : ℝ => 2 * x) (□ 3) (□ 6)`. Continuity-at-a-point is definitionally `ContinuousAt f x ↔ Tendsto f (□ x) (□ (f x))`.

```
-- A continuous function composed with a continuous function is continuous
example {α β γ : Type*} [TopologicalSpace α] [TopologicalSpace β] [TopologicalSpace γ]
  (f : α → β) (g : β → γ) (hf : Continuous f) (hg : Continuous g) :
  Continuous (g ∘ f) := by
  exact hg.comp hf
```

Honest caveat for this section: topology proofs lean heavily on knowing the *right* Mathlib lemma name (`Continuous.comp`, `continuous_def`, `IsOpen.inter`, etc.) rather than on generic tactics — this is precisely the regime where `exact?/apply?/#loogle` stop being optional conveniences and become load-bearing. Expect this to be a rich source of “wrong lemma name” hallucinations once the agent starts working topology problems — worth flagging explicitly in the error taxonomy (`docs/resources.md §5`, and the future logging-system doc).

2.12 Worked examples — Algebra

Mathlib’s algebraic hierarchy (`Monoid`, `Group`, `Ring`, `Field`, ...) is built from typeclasses; a `Subgroup G` is a bundled structure (carrier set + closure proofs), and a group homomorphism $f : G \rightarrow^* H$ is a bundled function plus a proof it respects multiplication (`map_mul`).

```
-- Basic group identities, provable purely from the group axioms
example {G : Type*} [Group G] (a b : G) : (a * b)⁻¹ = b⁻¹ * a⁻¹ := by
  exact mul_inv_rev a b
```

```
example {G : Type*} [Group G] (a : G) : a * a⁻¹ = 1 := by
  exact mul_inv_cancel a
```

```
-- Homomorphisms preserve the identity and inverses
example {G H : Type*} [Group G] [Group H] (f : G →* H) (a : G) :
  f (a⁻¹) = (f a)⁻¹ := by
  exact map_inv f a
```

```
-- Subgroup membership is closed under the group operation
example {G : Type*} [Group G] (H : Subgroup G) (a b : G) (ha : a ∈ H) (hb : b ∈ H) :
  a * b ∈ H := by
  exact H.mul_mem ha hb
```

For concrete numeric rings ($\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, polynomial rings), reach for `ring/ring_nf` first — it closes anything that follows from ring axioms alone without needing a single named lemma:

```
example (a b : Int) : (a + b) * (a - b) = a^2 - b^2 := by ring
```

Recurring pattern across all three topic areas: generic tactics (`ring`, `omega`, `simp`, `tauto`, `ext`) get you very far in set theory and basic algebra, where the underlying structure is “compute and compare.” Topology (and deeper algebra — quotients, isomorphism theorems) leans much more on *finding the specific lemma that encodes the mathematical fact you need*, which is a fundamentally different skill — and, again, plausibly a different hallucination profile for the agent to exhibit. This distinction should directly inform how the problem set (`docs/resources.md`, Goal 9) is difficulty-tiered.

2.13 Reading errors and diagnosing failed proofs

Lean’s error messages are the raw signal the agent harness will parse programmatically (see the pipeline chapter), so it’s worth learning to read them precisely rather than pattern-matching on vibes.

“type mismatch” — the term you supplied has a different type than the goal expects. Almost always from `exact`, or an `apply` where unification failed silently upstream. Read the *expected* vs. *got* types Lean prints; the mismatch is usually one argument, one implicit, or a coercion away.

“unknown identifier / unknown constant” — you referenced a lemma name that doesn’t exist (wrong name, wrong namespace, or it’s simply not the name Mathlib uses). This is the single most common *hallucination* failure mode for an LLM proposing tactics: confidently citing a plausible-sounding lemma (`Nat.mul_comm_left` when the real name is `Nat.mul_left_comm`, say) that was never in the library. `exact?/apply?/#loogle` exist specifically to route around needing to *remember* exact names.

“failed to synthesize instance” — a typeclass Lean needs (e.g. `TopologicalSpace  $\alpha$` , `DecidableEq  $\alpha$`) isn’t available in context. Usually means a hypothesis or `variable` declaration is missing, not that the proof strategy is wrong.

“unsolved goals” at the end of a `by` block — the tactic sequence ran out without closing everything. Check the last-printed goal state; it tells you exactly what’s left.

A goal that doesn’t shrink after a tactic that “succeeded” (no error, but the goal looks identical) — almost always `rw/simp` finding nothing to rewrite, silently doing nothing rather than erroring. Worth explicitly distinguishing from a genuine error in the agent’s logs: *tactic accepted but made no progress* is a distinct category from *tactic rejected*, and both are distinct again from *tactic accepted*, *goal changed*, *but agent is now further from a proof than before*.

sorry compiles. A file with `sorry` anywhere in a proof still type-checks and reports no errors — Lean treats `sorry` as “trust me.” This is the sharpest trap for automated pipelines: a proof search that terminates because the agent inserted `sorry` looks, from the outside, identical to success unless you explicitly grep for it. Any harness built on top of this tutorial’s material **must** treat `sorry/admit` as a distinct terminal state, not a success.

This taxonomy — wrong-term, wrong-name, missing-instance, incomplete, no-progress, silently-admitted — is a first pass at exactly the category system the project’s core statistical question depends on (`docs/resources.md` §5–6). It should be revisited and refined once real agent transcripts exist to check it against.

2.14 From tactics to the agent pipeline

Everything above described tactics as something *you* type in VS Code, watching the Infoview update live. The agent harness replaces that human loop with a programmatic one: the agent receives a serialized goal

state (context + goal, roughly what’s printed above the $\vdash$ in every example in this document), proposes one tactic string, and receives back either the next goal state or one of the error categories from §13 — with no Infoview, no hover tooltips, no `exact?` search run interactively (though the agent can still *issue* `exact?` as a tactic and read its output).

Concretely, this means the command-line invocation from the Lean 4 Environment Setup chapter, §8 (`lake env lean file.lean`, or the Lean REPL / language server protocol) is the actual interface the agent talks to — not VS Code. Every tactic in this tutorial is a string the agent must produce *syntactically correctly* before Lean even attempts to check whether it’s *semantically* valid, which means malformed syntax (missing `import`, wrong bracket type, a `Mathlib`-only tactic invoked without the import) is itself a distinct failure mode, prior to and separate from a mathematically wrong tactic.

The next documents in this project build directly on this:

- The Agent $\leftrightarrow$ Lean 4 Pipeline chapter (Goal 6) — the exact protocol between agent and Lean, and how goal states get serialized/parsed.
- The agent-thinking logging design (Goal 8) — what gets recorded at each of these steps, building on the error taxonomy in §13.
- The problem set (Goal 9) — segmented using exactly the topic split in §10–12 (set theory / topology / algebra), difficulty-tiered partly along the “generic tactics suffice” vs. “requires exact lemma recall” axis observed in §12.

2.15 Mathlib conventions: naming, dot notation, anonymous constructors

Section 13 flagged “wrong lemma name” as a major hallucination category once problems require real Mathlib recall (topology and deeper algebra especially). It’s worth knowing that Mathlib names are **not arbitrary** — they follow a documented, learnable scheme. Internalizing it (for you and, eventually, as prior knowledge encoded into the agent’s prompting) turns “guess the name” into “derive the name.”

2.15.1 Capitalization signals the kind of thing

1. **Proofs and theorem names** (terms of a `Prop`) use `snake_case`: `add_comm`, `mul_zero`, `lt_of_le_of_ne`.
2. **Types, structures, classes** use `UpperCamelCase`: `Group`, `TopologicalSpace`, `Subgroup`.
3. **Everything else** (functions, other data) uses `lowerCamelCase`.

So seeing `Nat.add_comm` (`snake_case` after the dot) tells you it’s a proof; seeing `TopologicalSpace` (`UpperCamelCase`) tells you it’s a class before you’ve read anything else about it.

2.15.2 The symbol to word dictionary

Names are built by spelling out the symbols in the statement. The core dictionary (abbreviated from the official guide):

Symbol	Word	Symbol	Word	Symbol	Word
	or		and	$\neg$	not
$\rightarrow$	of / (often omitted)		iff		exists
$=$	eq (often omitted)	$\neq$	ne		comp
	mem		union		inter
	(usually spelled subset)		sdiff		compl
$+$	add	- (binary)	sub	- (unary)	neg
$*$	mul	$^$	pow	1	inv
$\leq / <$	le/lt (or ge/gt)		dvd	$\Sigma /$	sum/prod

Knowing this, a name like `mul_left_cancel` is readable on sight: “multiplication, left [argument], cancel[ation].” `not_le_of_gt` reads as “not ($\leq$), given ($>$)” — matching its actual statement $a < b \rightarrow \neg b \leq a$. This is exactly the skill that turns `exact?/#loogle` from a crutch into a check.

2.15.3 Hypothesis ordering: “of”

When a name encodes hypotheses, they’re listed via `_of_` in the order they appear in the statement, not reversed: a theorem $A \rightarrow B \rightarrow C$ is named `C_of_A_of_B`. `lt_of_le_of_ne` therefore states something like $a \leq b \rightarrow a \neq b \rightarrow a < b$ — conclusion first, then each hypothesis in argument order.

2.15.4 $\leq / <$ vs $\geq / >$

Mathlib almost always states things with $\leq / <$, never $\geq / >$, but still uses `ge/gt` in *names* in a few specific situations: when two occurrences of the same relation in one name need to be told apart (`le/lt` for the first, `ge/gt` for a swapped second), when matching another relation’s argument order, or when describing an argument-swapped version. If you’re hunting for a lemma and `_le_` isn’t found, try `_ge_` before giving up.

2.15.5 Dot notation and the anonymous constructor

`h.left`, `h.mp`, `f.comp g` — a leading dot after a term calls a function in that term’s namespace with the term as first explicit argument of matching type. And `.left : P  $\wedge$  Q  $\rightarrow$  P` becomes `h.left` when `h : P  $\wedge$  Q`. This is why `Continuous.comp`, seen in §11, can be written `hg.comp hf`.

The anonymous constructor `{...}` builds a term of a structure/inductive type by position, letting Lean infer *which* constructor from the expected type — this is what’s happening every time you write `(n, hn)` to prove an `n`, or `(hp, hq)` to prove a `h`.

2.15.6 Structural suffixes worth recognizing

- **.ext** — a lemma of shape $(\forall x, f x = g x) \rightarrow f = g$; the thing `ext` (§8) invokes.
- **.ext_iff** — the bidirectional version, $f = g \leftrightarrow \forall x, f x = g x$.
- **.inj / _injective / _inj** — injectivity; `.inj` is usually a unidirectional, auto-generated constructor fact, `_inj` a bidirectional iff, `_injective` a full `Function.Injective f` statement.
- **Is-prefixed classes** (`IsOpen`, `IsClosed`, `IsTopologicalRing`) — a `Prop`-valued class describing an extra property of data you already have, as opposed to extra data itself.

Full reference (symbol dictionary, capitalization rules, structural-lemma conventions, more examples): [Mathlib naming conventions](#), cataloged in `docs/resources.md`.

2.16 Exercises (with worked solutions)

Work these before moving to the pipeline docs — they exercise every tactic family from §2–9 and preview the topic split from §10–12. Solutions follow each prompt; cover them and attempt the proof first.

1. Core tactics. Prove $(P \ Q : \text{Prop}) \ (hpq : P \rightarrow Q) \ (hp : P) : Q$.

```
example (P Q : Prop) (hpq : P → Q) (hp : P) : Q := by
  exact hpq hp
```

2. Rewriting. Prove $(a \ b \ c : \text{Nat}) \ (h1 : a = b) \ (h2 : b = c) : a + 1 = c + 1$.

```
example (a b c : Nat) (h1 : a = b) (h2 : b = c) : a + 1 = c + 1 := by
  rw [h1, h2]
```

3. have/suffices. Prove $(n : \text{Nat}) \ (h : n > 10) : n > 0$ two ways: with `have`, then with `suffices`.

```
example (n : Nat) (h : n > 10) : n > 0 := by
  have h' : n > 10 := h
  omega
```

```
example (n : Nat) (h : n > 10) : n > 0 := by
  suffices h' : n > 10 by omega
  exact h
```

4. Induction. Prove $(n : \text{Nat}) : n < n + 1$ by induction (don't just reach for `omega` — practice using `ih`).

```
example (n : Nat) : n < n + 1 := by
  induction n with
  | zero => omega
  | succ m ih => omega -- ih available but omega doesn't need it here;
                      -- compare with a genuinely inductive fact like §5's add_zero'
```

(Note: this particular fact is linear and `omega` alone would close the top-level goal without induction — the exercise is in the induction mechanics: naming `ih`, understanding it's unused here because the fact doesn't recurse on n 's structure. Try `theorem double_ge (n : Nat) : n ≤ 2 * n` by induction for a case where `ih` is genuinely load-bearing.)

5. Case split + automation. Prove $(n : \text{Nat}) : n \% 2 = 0 \vee n \% 2 = 1$.

```
example (n : Nat) : n % 2 = 0 ∨ n % 2 = 1 := by omega
```

6. ring vs omega. Prove $(x \ y : \text{Int}) : (x + y) * (x + y) = x*x + 2*x*y + y*y$, and separately $(a \ b : \text{Nat}) \ (h : a \leq b) : a + 3 \leq b + 3$. Identify which tactic fits which.

```
example (x y : Int) : (x + y) * (x + y) = x*x + 2*x*y + y*y := by ring
example (a b : Nat) (h : a ≤ b) : a + 3 ≤ b + 3 := by omega
```

7. by_contra. Prove $(n : \text{Nat}) \ (h : \neg n = 0) : n > 0$.

```
example (n : Nat) (h : ¬ n = 0) : n > 0 := by
  by_contra hc
  push_neg at hc
  omega
```

8. calc. Prove $(a\ b\ c : \text{Nat})\ (h1 : a \leq b)\ (h2 : b \leq c) : a \leq c$ with an explicit `calc` chain (not just `exact le_trans h1 h2`).

```
example (a b c : Nat) (h1 : a ≤ b) (h2 : b ≤ c) : a ≤ c := calc
  a ≤ b := h1
  _ ≤ c := h2
```

9. rcases/obtain. Prove $(h : \exists n : \text{Nat}, n > 3 \wedge n < 6) : \exists n : \text{Nat}, n > 0$.

```
example (h : ∃ n : Nat, n > 3 ∧ n < 6) : ∃ n : Nat, n > 0 := by
  obtain ⟨n, hn1, hn2⟩ := h
  exact ⟨n, by omega⟩
```

10. Set theory. Prove $(A\ B\ C : \text{Set } \alpha) : A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ — you saw this in §10; write it here from a blank slate.

```
example (A B C : Set α) : A ∩ (B ∪ C) = (A ∩ B) ∪ (A ∩ C) := by
  ext x
  simp only [Set.mem_inter_iff, Set.mem_union]
  tauto
```

11. Set theory, subset chain. Prove $(A\ B\ C\ D : \text{Set } \alpha)\ (h1 : A \subseteq B)\ (h2 : B \subseteq C)\ (h3 : C \subseteq D) : A \subseteq D$.

```
example (A B C D : Set α) (h1 : A ⊆ B) (h2 : B ⊆ C) (h3 : C ⊆ D) : A ⊆ D := by
  intro x hx
  exact h3 (h2 (h1 hx))
```

12. Algebra. Prove $\{G : \text{Type}^*\} [\text{Group } G]\ (a\ b\ c : G)\ (h : a * b = a * c) : b = c$ (left cancellation) using `mul_left_cancel` or by hand via `mul_left_cancel`₀-style reasoning — try `exact`? first if the exact name isn't obvious.

```
example {G : Type*} [Group G] (a b c : G) (h : a * b = a * c) : b = c := by
  exact mul_left_cancel h
```

13. Combinators. Prove $(b1\ b2 : \text{Bool}) : (b1 \&\& b2) = (b2 \&\& b1)$ by casing on both booleans and closing every resulting goal with one combinator call.

```
example (b1 b2 : Bool) : (b1 && b2) = (b2 && b1) := by
  cases b1 <=> cases b2 <=> rfl
```

14. Diagnose the error. The following proof fails. Without running it, predict which error category from §13 it hits, then fix it.

```
example (n : Nat) : n + 0 = n := by
  exact Nat.add_zer n -- typo: should be Nat.add_zero
```


Diagnosis: unknown identifier — `Nat.add_zer` doesn't exist (missing final `o`). This is exactly the “plausible-sounding but nonexistent name” hallucination flagged in §13; the fix is either correcting the typo or reaching for `exact?`.

15. Cross-topic capstone. Prove that for a continuous function $f : \mathbb{R} \rightarrow \mathbb{R}$ and a set S that's the preimage of an open set under f , S is open — i.e., restate and prove `Continuous f → IsOpen U → IsOpen (f-1 U)` using the definitional unfolding from §11.

```
example (f : ℝ → ℝ) (hf : Continuous f) (U : Set ℝ) (hU : IsOpen U) :
  IsOpen (f-1 U) := by
  exact hf.isOpen_preimage U hU
```

(If the exact lemma name here doesn't resolve in your environment, this is a live example of the topology-specific “lemma recall” difficulty flagged in §12 — use `exact?` and cross-check against `Mathlib.Topology.Continuous` in `docs/resources.md`.)

2.17 References

- [Theorem Proving in Lean 4](#) — Lean FRO / Lean community
- [Lean 4 Dev — Tactics course](#) — primary source for syntax and examples throughout this document
- [Mathematics in Lean](#) — Jeremy Avigad, Patrick Massot
- [Mathlib4 tactic documentation](#)
- [Mathlib.Topology.Continuous](#), [Mathlib.Topology.Defs.Basic](#)

Full annotated bibliography, including the LLM/hallucination and agent-Lean literature that motivates this project: `docs/resources.md`.

Chapter 3

Lean 4 Tactic Cheat Sheet

Agentic Hallucinations — Independent Study — Quick Reference (4 pp.).

Sources: lean4.dev/tactics, [Theorem Proving in Lean 4](#), [Mathlib tactic docs](#). Full bibliography: `docs/resources.md`.

3.1 Reading a goal state

```
case succ
n : Nat
ih : n + 0 = n
⊢ n + 1 + 0 = n + 1
```

Everything above $\vdash$ is the **context** (hypotheses/variables in scope). What's after $\vdash$ is the **goal** — what you still need to prove. Multiple goals are numbered/separated; $\cdot$ (a centered dot) focuses on the first one.

Term mode vs. tactic mode: `theorem foo : P := proof_term` is term mode (you write the proof object directly). `theorem foo : P := by tac1; tac2` enters tactic mode — each tactic transforms the goal state until none remain. This project works almost entirely in tactic mode, since that's what the agent proposes one step at a time.

3.2 Core / structural — closing and opening goals

Tactic	Does	Example
<code>intro x</code>	Introduces a hypothesis/variable from $\rightarrow$ or $\forall$	<code>intro n hn</code>
<code>intro {a, b}</code> <code>intros</code>	Introduces + destructures (,) in one step Introduces everything possible, auto-named (avoid in final proofs — unreadable names)	<code>intro {hp, hq}</code> <code>intros</code>
<code>exact e</code>	Closes the goal with an exact proof term <code>e</code>	<code>exact Nat.add_comm a b</code>
<code>exact?</code>	Searches the library for a term that closes the goal	<code>exact?</code>
<code>apply e</code>	Unifies <code>e</code> 's conclusion with the goal; leaves premises as new goals	<code>apply Eq.trans</code>
<code>apply?</code> <code>assumption</code>	Searches the library for a lemma to <code>apply</code> Closes the goal if some hypothesis matches exactly	<code>apply?</code> <code>assumption</code>
<code>rfl</code>	Closes goals <code>true</code> by computation/definitional equality	<code>rfl</code>
<code>have h : T := proof</code>	States + proves an intermediate fact <code>h : T</code>	<code>have hq : Q := hpq hp</code>
<code>have h : T := by tac</code>	Same, proved with tactics	<code>have : a ≤ a + b := by omega</code>
<code>let x := val</code>	Local transparent definition (unfolds automatically)	<code>let m := n + n</code>
<code>suffices h : T by tac</code>	Replaces goal with <code>T</code> , using <code>tac</code> to show <code>T → original goal</code>	<code>suffices h : n ≤ n+n by omega</code>
<code>show T</code>	Restates the goal as a definitionally-equal <code>T</code> (for clarity)	<code>show x > 10</code>
<code>constructor</code>	Applies the goal type's constructor (splits , builds structures)	<code>constructor</code>
<code>use e / exists e</code>	Provides a witness for	<code>use 5</code>

have vs let: `have` produces an opaque proof (only its *type* matters downstream); `let` produces a transparent value (its *definition* is visible to `rfl`/unification). Use `have` for propositions, `let` for reusable computed values.

3.3 Rewriting & simplification

Tactic	Does	Example
<code>rw [h]</code>	Replaces LHS of <code>h : a = b</code> with RHS, first match only per pass , all matches per rewrite pass	<code>rw [Nat.add_zero]</code>
<code>rw [← h]</code>	Rewrites right-to-left	<code>rw [← h]</code>
<code>rw [h1, h2]</code>	Chains rewrites left to right	<code>rw [h1, h2]</code>
<code>rw [h] at h2</code>	Rewrites inside hypothesis <code>h2</code> instead of the goal	<code>rw [h1] at h2</code>
<code>nth_rewrite 1 [h]</code>	Rewrites only the <i>n</i> -th matching occurrence	<code>nth_rewrite 1 [Nat.add_comm]</code>
<code>simp</code>	Repeatedly applies the @[simp] lemma database until fixed point; often closes the goal outright	<code>simp</code>
<code>simp [h1, h2]</code>	<code>simp</code> + extra lemmas/hypotheses	<code>simp [Nat.add_comm]</code>
<code>simp only [l1, l2]</code>	Uses only the listed lemmas — reproducible, preferred in libraries	<code>simp only [Nat.add_zero]</code>
<code>simp_all</code>	Simplifies goal and every hypothesis, using hypotheses as rewrite rules too	<code>simp_all</code>
<code>simp?</code>	Runs <code>simp</code> , suggests the minimal <code>simp only [...]</code> that achieves the same	<code>simp?</code>
<code>simp at h</code>	Simplifies a hypothesis instead of the goal	<code>simp at h</code>
<code>dsimp</code>	Definitional-only simplification (unfolds, no lemma rewriting; never fails oddly)	<code>dsimp [myDef]</code>
<code>norm_num</code>	Proves/simplifies goals with concrete numerals	<code>norm_num</code>
<code>norm_cast</code>	/ Normalizes / pushes numeric-type coercions	<code>norm_cast</code>
<code>push_cast</code>	(<code>→ →</code> , etc.)	
<code>unfold f</code>	Unfolds a definition <code>f</code> by name	<code>unfold triple</code>

rw vs simp: `rw` = precise, one rule at a time, you name every step. `simp` = automatic, applies many rules, less predictable. Reach for `rw` when you know the exact steps; `simp` when the goal is “obviously” simplifiable and you don’t care about the trace.

3.4 Case analysis, induction, destructuring

Tactic	Does	Example
<code>cases x with c1 => ... c2 => ...</code>	Splits on constructors of x 's type	<code>cases n with zero => rfl succ m => omega</code>
<code>induction x with c1 => ... c2 ih => ...</code>	Proof by induction; each non-base case gets an induction hypothesis ih	<code>induction n with zero => rfl succ m ih => simp [ih]</code>
<code>rcases h with (pat)</code>	Recursive destructuring of h , nested pairs (Mathlib)	<code>rcases h with (n, hn)</code>
<code>obtain (pat) := h</code>	Same as <code>rcases</code> , “goal-first” phrasing; also works to introduce a fresh existence proof	<code>obtain (n, hn) := h</code>
<code>rintro (pat)</code>	<code>intro + rcases</code> pattern in one step	<code>rintro (a, b) (c, d)</code>
<code>left / right</code>	Picks a disjunct of an goal	<code>left</code>
<code>match x with pat => ...</code>	Pattern-match in tactic mode (alternative to <code>cases</code>)	<code>match n with 0 => rfl n+1 => omega</code>

rcases pattern syntax: $\langle p1, p2 \rangle$ destructs pairs / $\cdot$ $p1$ | $p2$ splits $\cdot$ `rfl` substitutes an equality immediately $\cdot$ `_` discards $\cdot$ `-` clears the hypothesis after use.

3.5 Automation / decision procedures

Tactic	Solves	Notes
<code>ring</code>	Polynomial identities over (semi)rings — $+$ $-$ $*$ $^$ by constants, no division/inequalities	requires <code>import Mathlib.Tactic.Ring</code>
<code>ring_nf</code>	Normalizes a ring expression without requiring it close the goal	good mid-proof simplification step
<code>field_simp</code>	Clears denominators in field expressions (run before <code>ring</code> if division present)	<code>field_simp; ring</code>
<code>omega</code>	Linear arithmetic over $/ : + - < \leq = \neq$, mult/div/mod by constants	fails on variable $\times$ variable (non-linear)
<code>linarith</code>	Linear (in)equalities from hypotheses, ordered fields	<code>linarith [extra_fact]</code> to feed in a fact
<code>nlinarith</code>	<code>linarith</code> + limited nonlinear preprocessing (products, squares)	<code>nlinarith [sq_nonneg x]</code>
<code>polyrith</code>	Finds polynomial proofs via Gröbner bases (calls an external service)	slower; last resort over <code>ring/nlinarith</code>
<code>decide</code>	Evaluates a Decidable proposition to true/false	can be slow/blow up — see <code>native_decide</code>
<code>native_decide</code>	Like <code>decide</code> , compiled native code — much faster, less trusted	use for large finite checks
<code>tauto</code>	Closes propositional-logic tautologies	<code>tauto</code>
<code>aesop</code>	General automated proof search over registered <code>@[aesop]</code> rules	<code>aesop</code> , or <code>aesop?</code> to see the found proof

Decision table: polynomial equality $\rightarrow$ `ring`. Division present $\rightarrow$ `field_simp` then `ring`. Linear Nat/Int $\rightarrow$ `omega`. Linear inequalities generally $\rightarrow$ `linarith`. Nonlinear inequalities $\rightarrow$ `nlinarith`.

Concrete numerals $\rightarrow$ `norm_num`. Definitional/computed equality $\rightarrow$ `rfl`.

3.6 Logic, contradiction, structuring

Tactic	Does	Example
<code>by_contra h</code>	Goal $P \rightarrow$ assume $h : \neg P$, prove <code>False</code>	<code>by_contra h</code>
<code>by_cases h : P</code>	Splits into $h : P$ and $h : \neg P$ branches	<code>by_cases h : n = 0</code>
<code>contrapose/contrapose!</code>	Turns goal $P \rightarrow Q$ into $\neg Q \rightarrow \neg P$ (the <code>!</code> variant also pushes negations)	<code>contrapose!</code>
<code>push_neg</code>	Pushes $\neg$ inward through quantifiers/connectives (De Morgan)	<code>push_neg at h</code>
<code>exfalso</code>	Changes any goal to <code>False</code> (use with a contradictory hypothesis)	<code>exfalso</code>
<code>absurd h hn</code>	Given $h : P$ and $hn : \neg P$, closes any goal	<code>exact absurd h hn</code>
<code>and_intros</code>	Splits a conjunction goal into all conjuncts	<code>and_intros</code>

3.7 Equality, extensionality, congruence

Tactic	Does	Example
<code>calc</code>	Step-by-step chain of equalities/inequalities, each justified separately	see below
<code>ext x</code>	Proves equality (functions, sets, structures) by showing components/behavior match — needs <code>Mathlib</code>	<code>ext x</code>
<code>funext x</code>	Function extensionality only, core Lean	<code>funext x</code>
<code>congr</code>	Reduces $f\ a = f\ b$ to $a = b$ (breaks equality into congruence subgoals)	<code>congr; exact h</code>
<code>symm</code>	Flips $a = b$ to $b = a$ in the goal	<code>symm</code>
<code>trans y</code>	Splits $a = c$ (or $a \leq c$ etc.) into $a = y$ and $y = c$	<code>trans b</code>

```
example (a b c d : Nat) (h1 : a = b) (h2 : b < c) (h3 : c ≤ d) : a < d := calc
  a = b := h1
  _ < c := h2
  _ ≤ d := h3
```

`calc` chains transitively across mixed relations ($=$, $<$, $\leq$) as long as they compose.

3.8 Combinators (gluing tactics together)

Combinator	Does
<code>t1 <;> t2</code>	Runs <code>t2</code> on every goal produced by <code>t1</code>
<code>· tac</code>	Focuses the first goal, closes it with <code>tac</code> or fails
<code>first t1 t2</code>	Tries <code>t1</code> ; if it fails, tries <code>t2</code>
<code>repeat tac</code>	Repeats <code>tac</code> until it fails
<code>all_goals tac</code>	Runs <code>tac</code> on every goal; fails if <code>tac</code> fails on any
<code>any_goals tac</code>	Like <code>all_goals</code> try <code>tac</code> , but fails only if <code>tac</code> succeeds nowhere
<code>(tac1; tac2)</code>	Sequences tactics without requiring the goal close by the end (unlike <code>·</code>)

3.9 Debugging & discovery

Tool	Use
<code>sorry</code>	Placeholder — admits the goal unproved (compiles, but marks the proof incomplete). Never in a final proof; critical signal in agent logs.
<code>#check e</code>	Shows the type of <code>e</code> — e.g. <code>#check Nat.add_comm</code>
<code>#eval e</code>	Evaluates <code>e</code> and prints the result — e.g. <code>#eval 1 + 1</code>
<code>exact? / apply?</code>	Search the library for a closing/applicable lemma
<code>simp?</code>	Suggests the minimal <code>simp only [...]</code>
<code>#loogle</code> , <code>#leansearch</code> , <code>#statesearch</code>	In-editor search of Mathlib by type/name/goal shape — e.g. <code>#loogle _ * (_ ^ _)</code>

3.10 Notation quick-typing (VS Code unicode input, backslash prefix)

Symbol	Type	Symbol	Type	Symbol	Type
	<code>\forall</code>		<code>\exists</code>	<code>¬</code>	<code>\not</code>
	<code>\and</code>		<code>\or</code>	<code>→</code>	<code>\to</code> or <code>\r</code>
	<code>\iff</code>		<code>\in</code>		<code>\notin</code>
	<code>\subseteq</code>		<code>\cap</code>		<code>\cup</code>
	<code>\N \Z \Q \R \C</code>		<code>\emptyset</code>		<code>\<</code> <code>\></code>
<code>≤ ≥ ≠</code>	<code>\le \ge \ne</code>		<code>\comp</code>		(goal turnstile, not typed)

3.11 Common gotchas

- **`rw` fails silently-ish** if the LHS pattern isn't syntactically present in the goal — check with `#check` on the lemma first.
- **`simp` can over-simplify** and leave a goal in an unexpected shape mid-proof; prefer `simp only` in anything meant to be stable.
- **`omega/ring` are decision procedures for *specific* fragments** — `omega` fails on any variable×variable term; `ring` fails on inequalities or division. Route to the right tool (see the automation table above).
- **`Nat` subtraction truncates** at 0 (`a - b = 0` when `a < b`) — `omega` knows this, manual reasoning often doesn't.
- **`have` is opaque, `let` is transparent** — swapping them can silently break a proof that relied on unfolding.
- **`sorry/admit` compile successfully** — always `grep` a file for `sorry` before treating it as done. (This is exactly the kind of silent failure the agent-hallucination logging in this project needs to catch — see `docs/resources.md` §5–6.)

Full worked examples and explanations: the Lean 4 Tutorial chapter. Tactic-by-tactic deep dives: lean4.dev/tactics.

Chapter 4

Minimum Viable Tactic Set

4.1 Why a restricted set at all

The harness (`agents/first_e2e_test.py`) already makes a real design decision on this: its system prompt explicitly bans `ring`, `omega`, `simp`, `aesop`, `tauto`, and `decide`, instructing the model to “prove this step by step using `intro/rw/exact/apply/calc` instead, so the proof trace is actually informative for the study.”

The reason is specific to what this project studies. A decision procedure like `ring` or `omega` either closes a goal outright or fails — it gives no partial signal about *which* reasoning step was wrong. Since this project’s subject is hallucinations in the agent’s step-by-step reasoning, a tactic that swallows several reasoning steps into one opaque call is close to useless as data: a failure only says “the decision procedure didn’t apply here,” not anything about the agent’s own reasoning. This chapter makes that already-made decision explicit and generalizes it into a usable reference, rather than inventing a new policy.

4.2 The floor set

The following tactics, all drawn from the Tactic Cheat Sheet chapter (§3), are the ones needed to make progress on a goal one legible reasoning step at a time. Nothing here is a decision procedure; each one corresponds to a single, inspectable move.

Category	Tactics	Role
Structural	<code>intro</code> , <code>exact</code> , <code>apply</code> , <code>have</code> , <code>rfl</code> , <code>constructor</code> , <code>use</code>	Open and close goals directly; no search or automation
Rewriting	<code>rw [h]</code> , <code>rw [← h]</code> , <code>nth_rewrite</code>	Replace one term with another, named explicitly by the agent
Case analysis / induction	<code>cases</code> , <code>induction</code> , <code>rcases</code> , <code>obtain</code>	Split a goal on structure; every branch stays visible
Equality & structuring	<code>calc</code> , <code>symm</code> , <code>trans</code> , <code>congr</code>	Chain justified steps explicitly
Logic / contradiction	<code>by_contra</code> , <code>by_cases</code> , <code>ex falso</code> , <code>push_neg</code> , <code>left</code> , <code>right</code>	Restructure the goal without deciding it
Goal management	<code>·</code> (<code>focus</code>), <code><</code> ; <code>></code>	Sequence and scope tactics across multiple goals

4.3 Explicitly excluded

Banned by the harness today: `ring`, `omega`, `simp` (all variants: `simp_all`, `simp_only`, `simp?`, `simp_at`), `aesop`, `tauto`, `decide`.

Not currently referenced by the harness's ban list, but the same family of decision procedure for the same reason — excluded here too, pending explicit reconsideration: `ring_nf`, `field_simp`, `linarith`, `nlinarith`, `polyrith`, `native_decide`.

4.4 Undecided

- `norm_num`, `norm_cast`, `push_cast` — narrower than `simp`, but still automated lemma search rather than a single named step. Not yet run against a real goal in this project, so not yet placed on either list.
- `dsimp` — unfolds definitions only, no lemma database; arguably closer to the floor set than to `simp` despite the name. Same status: undecided until it's actually needed.

4.5 Discovery and debugging tools — not proof steps

`sorry`, `#check`, `#eval`, `exact?`, `apply?`, `simp?`, `#loogle`/`#leansearch`/`#statesearch` are all tools a human uses while writing or debugging a proof. None of them are tactics an agent should ever propose as a logged step: `sorry` admits the goal without proving it (exactly the kind of silent failure this study needs to catch, per the Tactic Cheat Sheet's gotchas section), and the `?`-suffixed tools search the library rather than performing an inspectable reasoning step themselves.

4.6 Empirical tracking

The floor set above is derived from the harness's existing design and the cheat sheet, not yet from a completed proof run — `agents/first_e2e_test.py` has not been executed yet (see `agents/README.md`, which flags an unverified exception-handling branch). The table below is the place to record, tactic by tactic, which of the above were actually exercised by a real proof and which turned out to be unnecessary.

Tactic	Needed for	Confirmed?

Chapter 5

Agent and Lean 4 Pipeline

Agentic Hallucinations — Independent Study. Design spec for how the agent and Lean 4 actually communicate, and what “environment switching” means across this project’s full stack. Written to be concrete enough to hand to an implementer (human or Claude Code) as a blueprint — not just a description.

Sources: [leanprover-community/repl](#) (protocol fetched and quoted directly below), [Pantograph \(arXiv:2410.16429\)](#), [lean-lsp-mcp](#), [LeanInteract](#). Full bibliography: [docs/resources.md](#).

5.1 The loop, restated precisely

From the original project brief: the agent is given a theorem, proposes one tactic, Lean verifies it, and either returns a new goal state or an error — repeat until the goal list is empty (proved) or some stopping condition is hit (budget exhausted, agent gives up, non-termination).

Stated as a state machine:

```
state = (theorem, goal_stack, history)
loop:
  agent.observe(goal_stack, history) → tactic_string
  result = lean.apply(tactic_string, goal_stack)
  if result.ok and result.goals == []:
    state = PROVED; stop
  elif result.ok:
    goal_stack = result.goals
    history.append((tactic_string, "success", result.goals))
  else:
    history.append((tactic_string, "error", result.error_message))
    # goal_stack unchanged – agent tries again from the same state,
    # now with the error message added to what it can see
  if budget_exhausted or agent_signals_stop:
    state = FAILED; stop
```

Everything in this document is about making `lean.apply(...)` a real, callable thing, and about what `goal_stack` and `result` actually look like on the wire — because *that* representation is what determines what the agent can see, and therefore what “hallucination” even means operationally (proposing a tactic that’s invalid given what it could see, vs. one that’s invalid period).

5.2 Candidate interfaces, surveyed

There is no single official “Lean API for agents” — four real options exist, each with different tradeoffs:

Interface	How it works	Strengths	Weaknesses
File + lake env lean (from the Environment Setup chapter, §8)	Write a whole <code>.lean</code> file, run <code>lake env lean file.lean</code> , parse stdout/stderr	Zero extra dependencies; exactly what CI and the setup doc already verify	No incremental state — every attempt re-elaborates the whole file from scratch; painfully slow for a multi-step tactic loop; output is meant for humans, not machines
leanprover-community/repl	Persistent process, JSON in/out over stdin/stdout, <code>proofState</code> integers track state incrementally	Purpose-built for exactly this; incremental (no re-elaboration per tactic); structured JSON; supports backtracking via reusing an earlier <code>proofState/env</code> id	“Experimental” tactic mode per its own README; one Lean process = one project’s environment, so multi-project setups need multiple processes (see §5)
Lean language server (LSP) , via <code>lake serve</code> or wrapped as <code>lean-lsp-mcp</code>	The same server VS Code talks to; JSON-RPC 2.0; query goal state at a file position	Reuses infrastructure that’s already battle-tested by every human Lean user; also exposes hover/definition/diagnostics beyond just tactic state	Designed around a <i>live-edited file</i> , not a clean append-only tactic stream — mapping “propose one tactic” onto “edit this file at this position” is more indirect than the REPL’s native <code>{"tactic": ..., "proofState": ...}</code> shape
Pantograph	Purpose-built ML-facing API/REPL, supports tree search (MCTS) natively	Most feature-rich for search-heavy agent designs; used in published LLM+Lean pipelines (Draft-Sketch-Prove)	Heavier dependency, smaller community than the REPL above; overkill if the agent is doing simple sequential tactic proposal rather than tree search

Recommendation for this project: the `leanprover-community/repl`, wrapped in Python (directly, or via `LeanInteract`). It matches the project’s stated loop shape almost exactly (see the tactic-mode example below), is incremental (fast — no re-elaborating the whole file per tactic), and is simple enough to reason about and log precisely, which matters more here than raw proof-search power. Revisit Pantograph if/when the harness grows a tree-search or backtracking-heavy agent design rather than the linear one-tactic-at-a-time loop described in the project brief.

5.3 The REPL protocol, concretely

Run via `lake exe repl` (or, pointed at a specific project’s dependencies: `lake env /path/to/repl/.lake/build/...`). Communicates via **one JSON object per command, blank-line separated**, on stdin, with one JSON object back on stdout.

5.3.1 Starting a theorem (command mode to tactic mode)

You open tactic mode by sending a command containing a `sorry`; the response gives you a `proofState` id to start from:

```
// → stdin
{"cmd": "theorem my_thm (x : Unit) : Nat := by sorry"}

// ← stdout
{"sorries":
  [{"proofState": 0,
    "pos": {"line": 1, "column": 29},
    "goal": "x : Unit\n⊢ Nat",
    "endPos": {"line": 1, "column": 34}}],
  "messages":
  [{"severity": "warning", "pos": {...}, "endPos": {...},
    "data": "declaration uses 'sorry'"}],
  "env": 0}
```

goal here — `"x : Unit\n⊢ Nat"` — is exactly the goal-state text this project's tutorial (the Lean 4 Tutorial chapter, §1) teaches how to read. This is the string (or a structured equivalent) the agent should be shown at each step.

5.3.2 One tactic step

```
// → stdin
{"tactic": "apply Int.natAbs", "proofState": 0}

// ← stdout
{"proofState": 1, "goals": ["x : Unit\n⊢ Int"]}

// → stdin
{"tactic": "exact -37", "proofState": 1}

// ← stdout
{"proofState": 2, "goals": []}
```

`"goals": []` is the terminal success state — the theorem is proved. This is the exact shape of `lean.apply()` from §1: input is `{tactic, proofState}`, output is a new `proofState` plus the resulting `goals` list (empty on success).

5.3.3 The error case

An invalid tactic doesn't crash the process — it returns a `messages` array with `"severity": "error"` and the proof state does **not** advance (you're still at the same `proofState`, free to try again):

```
// → stdin
{"tactic": "exact 37", "proofState": 1} // wrong: goal wants Int, not the literal misapp

// ← stdout (schematic – exact message text depends on the specific mismatch)
{"proofState": 1,
  "messages": [{"severity": "error", "data": "type mismatch ..."}]}
```

This is the critical mechanic for the hallucination analysis: **a rejected tactic does not lose progress** — the agent is still at `proofState 1` and can retry. So “how many iterations to converge” (from the project’s core question) is naturally just *the count of tactic-submission round-trips*, separable into accepted-and-advanced vs. rejected-and-retried, per `proofState`.

5.3.4 Backtracking and persistence

Any earlier `env` or `proofState` integer can be reused as the starting point for a new command — this *is* backtracking, for free, without re-running anything. States can also be pickled to `.olean` files and restored later (`{"pickleTo": "path", "proofState": n} / {"unpickleProofState-From": "path"}`), which matters for §6 (long-running logging) and for resuming an interrupted batch run.

5.4 Data model: what gets exchanged per step

Distilling the above into the fields the harness needs to track per iteration:

Field	Source	Notes
<code>theorem_id</code>	problem set (<code>docs/resources.md</code> topic/difficulty split, Goal 9)	which problem this attempt belongs to
<code>attempt_id</code>	harness	one per full proof attempt (a theorem may be attempted multiple times)
<code>step_index</code>	harness	position in this attempt’s tactic sequence
<code>proofState_before</code>	REPL	the <code>proofState</code> id the tactic was proposed against
<code>tactic_string</code>	agent output	exactly what the agent proposed, verbatim — no normalization before logging
<code>proofState_after</code>	REPL	new id if accepted; unchanged if rejected
<code>goals_before</code> / <code>goals_after</code>	REPL goal/goals field	the actual text shown to / returned to the agent
<code>outcome</code>	derived	advanced / rejected / proved (<code>goals == []</code>) / no_progress (accepted syntactically but <code>goals_after == goals_before</code>)
<code>error_message</code>	REPL messages	full text, only present on <code>outcome == rejected</code>
<code>wall_time_ms</code>	harness	time for this one REPL round-trip — needed to separate “slow tactic” (e.g. a heavy <code>simp/aesop</code> call) from a hung process (§7)

This table is the seed of the logging schema for Goal 8 (agent-thinking logging system) — that doc should extend this with *why* the agent chose the tactic (its reasoning/prompt context), not just what Lean did with it.

5.5 Environment switching — three distinct senses

“Environment switching” turns out to mean three different things at three different layers. Conflating them is an easy source of bugs once the harness is running many theorems across topics.

5.5.1 Toolchain/project switching

Each Lean **project** pins a `lean-toolchain` (exact compiler version) and a `lakefile` dependency set (which Mathlib commit, which other libraries). A single REPL process is started via `lake env <repl-binary>` **inside a specific project directory** and inherits *that* project’s toolchain and imports for its whole lifetime.

For this project’s problem set (set theory / topology / algebra, all drawn from Mathlib per the Lean 4 Tutorial chapter, §10–12), **one Mathlib-linked project should suffice for all three topics** — Mathlib is a single library spanning all of them, so topic segmentation is a *content/selection* concern, not an environment concern. Toolchain switching only becomes real if the problem set later pulls in an external benchmark repo with its own pinned toolchain (e.g. cloning `miniF2F` or `PutnamBench` directly rather than re-stating their theorems inside our own project — see `docs/resources.md` §3). In that case, the harness needs one REPL process (and one working directory) per distinct toolchain, and a router that dispatches each theorem attempt to the correct process based on which benchmark it came from.

5.5.2 Session/state switching

Within a single REPL process, each theorem attempt starts from a fresh `env/proofState` (§3). Retrying a theorem, or exploring two different tactic choices from the same point, is just reusing an earlier `proofState` id rather than starting over — cheap, and exactly what makes backtracking-capable agent designs (or simply “let me log both the accepted path and the road not taken”) tractable. The harness needs a clear policy here: does a new *attempt* at the same theorem get a fresh `proofState 0`, or does it resume from wherever the last attempt got stuck? (Recommendation: fresh state per attempt, logged as a new `attempt_id` — resuming from a stuck point conflates “the agent tried again” with “the agent kept going,” which are different statistical events.)

5.5.3 Interface switching (human vs. machine)

This is the sense closest to the original phrasing of Goal 7 (the not-yet-written fluency guide) and to the Environment Setup chapter, §8: VS Code + the Lean 4 extension is how *you* build, debug, and sanity-check problems interactively (Infoview, hover, `exact?` suggestions rendered inline); the REPL (or LSP-via-MCP) is how the *harness* talks to Lean headlessly, with no human in the loop. A theorem should generally be authored and manually verified once in VS Code (confirm it’s actually provable, get a feel for its difficulty) before it’s added to the problem set the harness runs against — the REPL is not a good place to *discover* whether a theorem statement itself is even correct, only to run an already-trusted theorem through repeated automated attempts.

5.6 Failure modes at the plumbing level

Distinct from the mathematical error taxonomy in the Lean 4 Tutorial chapter, §13 (wrong term, wrong lemma name, missing instance, incomplete, no-progress, silent `sorry`), the harness also needs to handle failures in the *plumbing itself* — these are infrastructure bugs, not agent hallucinations, and conflating the two categories would corrupt the statistical analysis:

- **Malformed JSON / protocol violation** — the agent (or a bug in the harness) sends something the REPL can’t parse. Should be caught and logged as a harness error, never attributed to the agent’s mathematical reasoning.

- **Process crash** — the REPL process dies mid-session (e.g. Mathlib bug, memory exhaustion). Needs a restart-and-resume policy; whatever attempt was in flight should be marked `incomplete_infra_failure`, not `failed`.
- **Non-terminating tactic** — some tactics (`simp` with a bad lemma set, certain `aesop` configurations) can hang rather than error. The harness needs a **per-tactic timeout** with a hard kill, and that timeout event is itself a data point worth logging (which tactics the agent proposes that are prone to hanging is exactly the kind of pattern the hallucination analysis wants to catch).
- **Resource exhaustion across a long run** — REPL processes accumulate `Environment/proofState` objects in memory; a long batch run needs periodic process restarts (using the pickling mechanism from §3 to preserve any state worth keeping) rather than one REPL process for the entire experiment.

5.7 Open design decisions

Flagging these explicitly rather than silently deciding them — they affect the statistics, so they’re worth deciding deliberately once the logging doc (Goal 8) and problem set (Goal 9) exist alongside this one:

1. **What counts as “one iteration”?** A single `{"tactic": ...}` call, or a semicolon/`<>`-chained combinator treated as one agent turn (per the Lean 4 Tutorial chapter, §9’s note on this same ambiguity)? Affects every downstream “iterations to converge” statistic.
2. **Retry budget policy.** Fixed max tactics per theorem? Max wall-clock time? Does the agent get to see the full history of its own rejected attempts, or only the current goal state?
3. **What exactly gets shown to the agent on error** — the raw Lean error string (verbose, sometimes includes internal term representations) or a cleaned/truncated version? This is a real methodological choice: a cleaner error message could materially change the agent’s hallucination rate, which means it needs to be fixed and reported, not tuned ad hoc per run.
4. **Attempt independence.** Are repeated attempts at the same theorem statistically independent draws (fresh `proofState 0`, no memory of prior attempts) or is the agent allowed to see its own history across attempts? Both are defensible, but they answer different questions, and the analysis needs to know which one was run.

5.8 What this doc feeds

- **Goal 8 (agent-thinking logging system)** extends §4’s per-step schema with the agent’s actual reasoning trace (prompt, chain-of-thought if any, model/temperature metadata) — this doc defines the *Lean-side* half of each logged record; Goal 8 defines the *agent-side* half.
- **Goal 9 (problem set)** should be authored and hand-verified via the VS Code path (§5c) before being fed to the harness described here.
- **Goal 7 (environment-switching fluency guide)** is the practitioner-facing companion to §5 — this doc explains the *architecture* of the three switching senses; Goal 7 should be the concrete day-to-day checklist (“here’s how you personally move between VS Code, a terminal REPL session, and reading harness logs without losing track of which project/state you’re in”).

5.9 References

- [leanprover-community/repl](#) — protocol and JSON examples in §3 quoted directly from this README
- [Pantograph: A Machine-to-Machine Interaction Interface for Advanced Theorem Proving, High Level Reasoning, and Data Extraction in Lean 4](#) — Aniva, Sun, Miranda, Barrett, Koyejo (arXiv:2410.16429)
- [lean-lsp-mcp](#)
- [LeanInteract](#) — Python wrapper, likely implementation choice once past design stage
- The Lean 4 Tutorial chapter, §13 (mathematical error taxonomy, distinct from §6’s infrastructure failures here)
- Full annotated bibliography, including the agent-Lean systems this design was compared against (LeanDojo/ReProver, LeanAgent, Prover Agent, APOLLO): [docs/resources.md](#)

Chapter 6

Mobile-Capable LLMs for Mathematical Reasoning

This chapter surveys which language models are actually practical to run on a phone for mathematical reasoning tasks, then argues for a specific model to use in our hallucination study.

6.1 The landscape (2026)

Two families of models are viable on-device today, and they trade off differently.

General-purpose edge models. Phi-4-mini (3.8B) and Gemma 3 4B are the two most-cited on-device options. Phi-4-mini leads on reasoning quality — 88.6% on GSM8K — and runs at roughly 13–18 tokens/sec on an iPhone 17 Pro, 10–15 tok/s on an iPhone 16 Pro, once quantized.¹ Gemma 3 4B is the faster of the two (around 27 tok/s on iPhone 16 Pro via Google’s AI Edge SDK) and is generally rated the best all-around edge model, though its math-specific numbers are less standout than Phi-4-mini’s.² Both are general chat/instruction models that happen to be good at math, not models built for math.

Math-specialized small models. The Qwen2.5-Math family (0.5B/1.5B/7B/72B) is trained specifically for mathematical reasoning via a self-improvement pipeline, and is released with official GGUF quantizations for llama.cpp-style local inference.³ GLM-Z1-9B is another math-strong small model, but at 9B it sits at the edge of what’s comfortable on a phone rather than squarely inside it.⁴

Quantization. Q4_K_M (4 bits/weight) is the practical standard for mobile deployment in 2026, keeping roughly 95% of full-precision quality at a quarter of the file size.⁵ Across the board, 1–3B parameters is the comfortable range for typical phones; 7–9B only works on current flagship hardware.

6.2 Model choice: Qwen2.5-Math-1.5B-Instruct

We pick **Qwen2.5-Math-1.5B-Instruct** for this project.

What it is. A 1.5B-parameter member of the Qwen2.5-Math series, instruction-tuned and trained via an iterative self-improvement process described in the Qwen2.5-Math technical report.⁶ It ships with

¹<https://www.promptquorum.com/power-local-llm/mobile-llm-models-phi4-gemma-smollm>

²<https://awesomeagents.ai/leaderboards/edge-mobile-llm-leaderboard/>

³<https://github.com/QwenLM/Qwen2.5-Math>

⁴<https://www.siliconflow.com/articles/en/best-open-source-LLM-for-math>

⁵<https://www.siliconflow.com/articles/en/best-lightweight-LLMs-for-mobile-devices>

⁶<https://arxiv.org/pdf/2409.12122>

official GGUF weights, so it runs directly under llama.cpp / MLC-LLM without any conversion step.

Why.

- *It punches far above its parameter count on math specifically.* With tool-integrated reasoning it scores 79.7 on the MATH benchmark⁷ — competitive with, or ahead of, general models 2–3× its size on math tasks. That’s a stronger reasoning-per-parameter trade than either Phi-4-mini or Gemma 3 4B offers.
- *It is genuinely small.* At 1.5B and Q4 quantization, the memory footprint is roughly 1 GB, which runs on mid-range Android hardware, not just this year’s flagship iPhone. That matters if we want results that generalize past top-tier devices.
- *It’s a math specialist, not a generalist doing math.* Since this project studies hallucinations *in mathematical tasks*, a model trained specifically on math gives a cleaner read on math-specific failure modes, rather than mixing in general-instruction-following weaknesses.
- *Fully open and offline.* Open weights plus official GGUF builds mean we can run everything locally with no API dependency — important for reproducibility, since we’re not at the mercy of an upstream model being silently updated mid-study.
- *The self-improvement training story is itself interesting.* The model was partly trained on math data it generated and filtered itself. That’s a plausible source of distinctive hallucination signatures worth comparing against a purely human-data-trained baseline.

Caveat. The 79.7 MATH score relies on tool-integrated reasoning (code execution), not pure chain-of-thought; the CoT-only score is lower. Since our study cares about the model’s own reasoning failures, we should benchmark both modes ourselves rather than citing the TIR number as if it reflects unaided reasoning.

⁷<https://www.emergentmind.com/topics/qwen2-5-math-1-5b>

Chapter 7

The LLM We're Using

Agentic Hallucinations — Independent Study. A record of which model actually drives the agent harness, and where that choice lives in the code.

7.1 What the harness calls

The end-to-end harness script, `agents/first_e2e_test.py`, hardcodes the model as a module-level constant:

```
MODEL = "claude-sonnet-5"
```

It is called through the official `anthropic` Python SDK, using the Messages API:

```
import anthropic

client = anthropic.Anthropic() # reads ANTHROPIC_API_KEY from env

resp = client.messages.create(
    model=MODEL,
    max_tokens=100,
    system=SYSTEM_PROMPT,
    messages=[{"role": "user", "content": user_msg}],
)
```

The client is constructed with no arguments, so it resolves credentials from the environment — specifically `ANTHROPIC_API_KEY`, which the script's own docstring instructs you to set before running.

7.2 It is a constant, not configuration

`MODEL` is a plain module-level constant. It is not read from a config file, an environment variable, or a command-line argument, and nothing else in the repository overrides it. Changing which model the harness uses therefore means editing that line in `agents/first_e2e_test.py` directly.

7.3 Status: current state, not a final choice

The script's own docstring describes it as a *first draft* smoke test rather than production harness code, and flags at least one branch inside it as unverified pending a real run. What this chapter records is therefore the model in use as of the current state of the repository — not a settled decision about which model the study will ultimately run against.